BLOCK 33

Paths, Files, and Reading Text

Day 5 begins connecting Python to external data.

Python 101 · Day 5 Block 1

30 minutes teaching

+ 30 minutes exercises

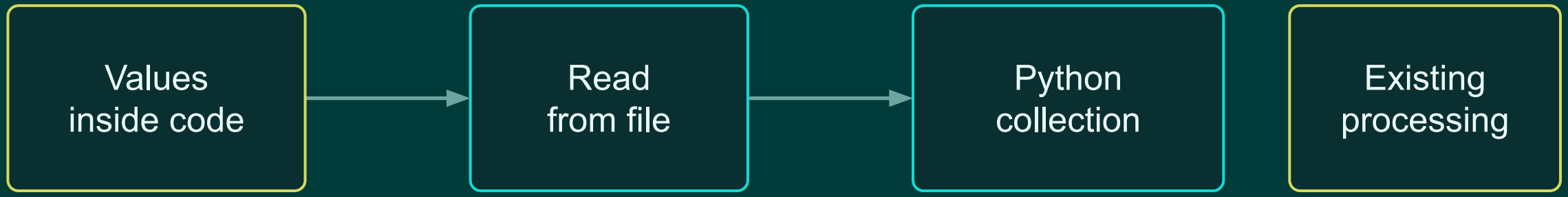
Today: Python stops using only data typed into code and starts reading real files.

BLOCK 33

The Day 5 Shift

Same processing skills. New data source.

Earlier days:




```
records = [
    "EQ-1001",
    "EQ-1002",
]
```

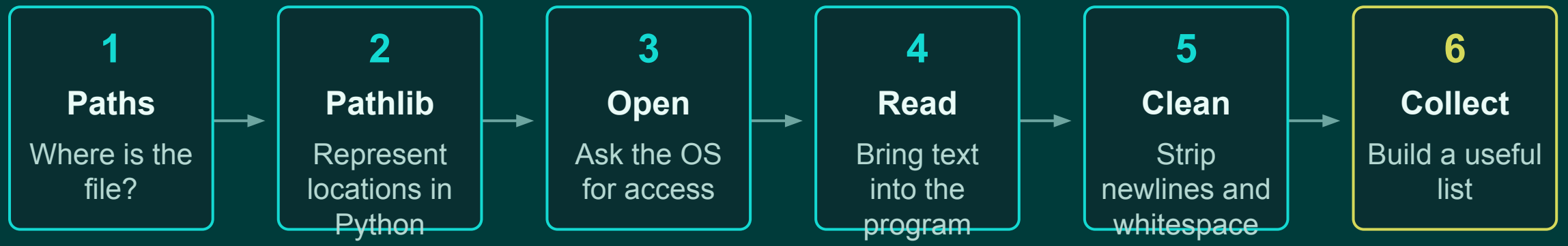
```
with open(file_path) as file:
    for line in file:
        record_ids.append(line.strip())
```

The processing logic can stay familiar. Only the input source changes.

BLOCK 33

Block Roadmap

A practical path from files to Python data.

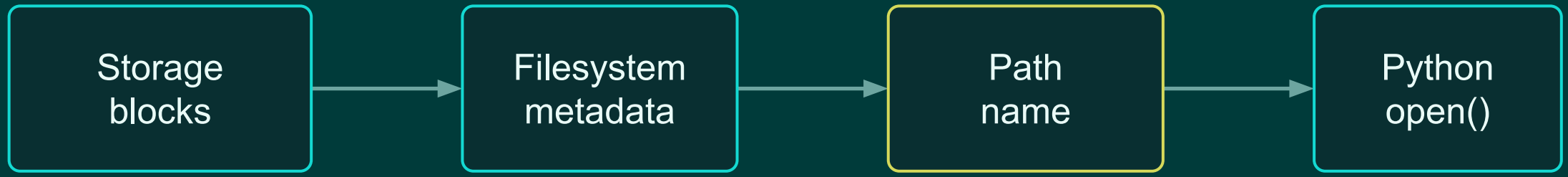


Goal by the end: read a messy text file, normalize records, separate valid and invalid IDs, and print a summary.

BLOCK 33

Revisiting the Filesystem

The operating system hides storage details behind names and paths.




```
data/raw/equipment.txt
```

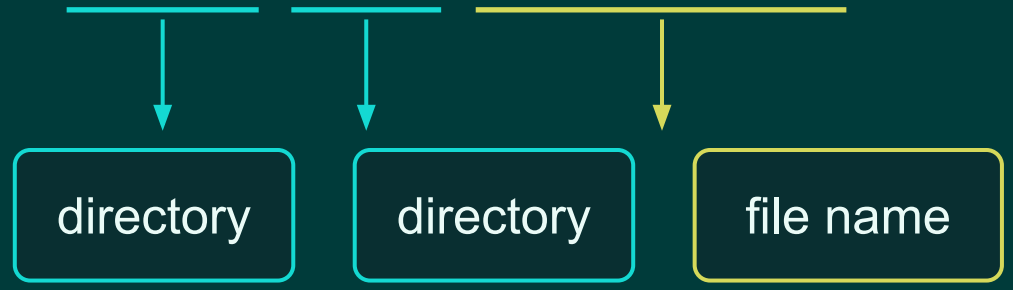
Python does not need to know the physical blocks. It asks the OS to open the file located at that path.

BLOCK 33

What a Path Communicates

A path is a structured description of a file location.

```
data/raw/equipment.txt
```



Mental model:

A path identifies a location. It does not automatically read the file.

BLOCK 33

Relative vs Absolute Paths

Use relative paths for classroom projects unless there is a reason not to.

ABSOLUTE PATH

```
/home/student/python101/data/equipment.txt
```

Starts from the filesystem root. Often tied to one computer or user.

RELATIVE PATH

```
data/equipment.txt
```

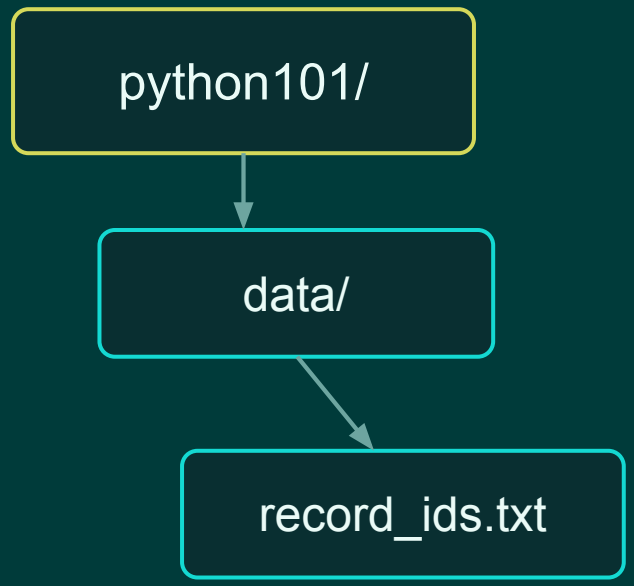
Starts from the current working directory. Easier to move with the project.

Practical rule: keep project data inside project folders, then use relative paths.

BLOCK 33

Current Working Directory

Relative paths depend on where the program is running from.



If the current working directory is:

```
python101/
```

then this works:

```
data/record_ids.txt
```

When a path “should work” but does not, check where Python is running from.

BLOCK 33

Introducing pathlib

Path objects represent locations cleanly and portably.

```
from pathlib import Path

file_path = Path("data/equipment.txt")
```

Path object
represents a location

Creating a Path does not open the file or read its contents.

BLOCK 33

Inspecting a Path

A path can answer questions before we open anything.

```
from pathlib import Path

file_path = Path("data/equipment.csv")

print(file_path.name)
print(file_path.suffix)
print(file_path.parent)
print(file_path.exists())
```

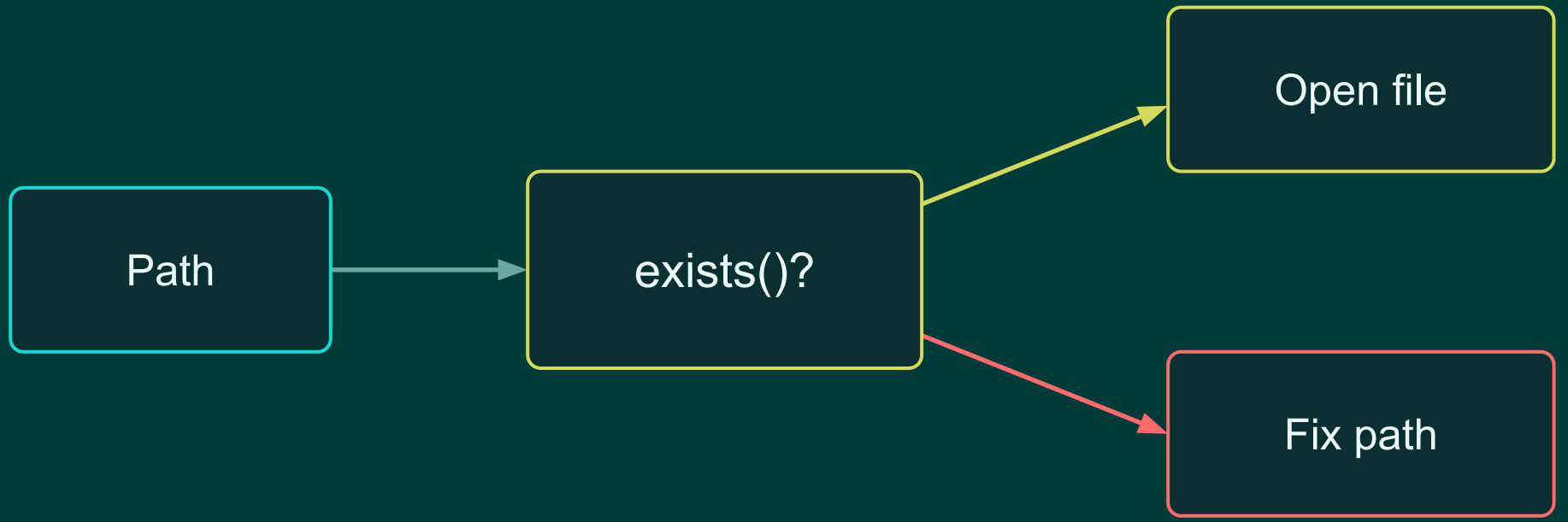
CONCEPTUAL OUTPUT

```
name:      equipment.csv

suffix:    .csv

parent:    data

exists:    True / False
```

BLOCK 33

Checking Before Opening

A missing file is often a path problem, not a Python mystery.




```
if file_path.exists():
    print("Found it")
else:
    print("Check the path")
```

BLOCK 33

Opening a Text File

The old pattern works, but it is easy to forget cleanup.

MANUAL OPEN / CLOSE

```
file = open("data/equipment.txt")

contents = file.read()

file.close()
```

RISK

If the program forgets file.close(), the resource may stay open longer than intended.

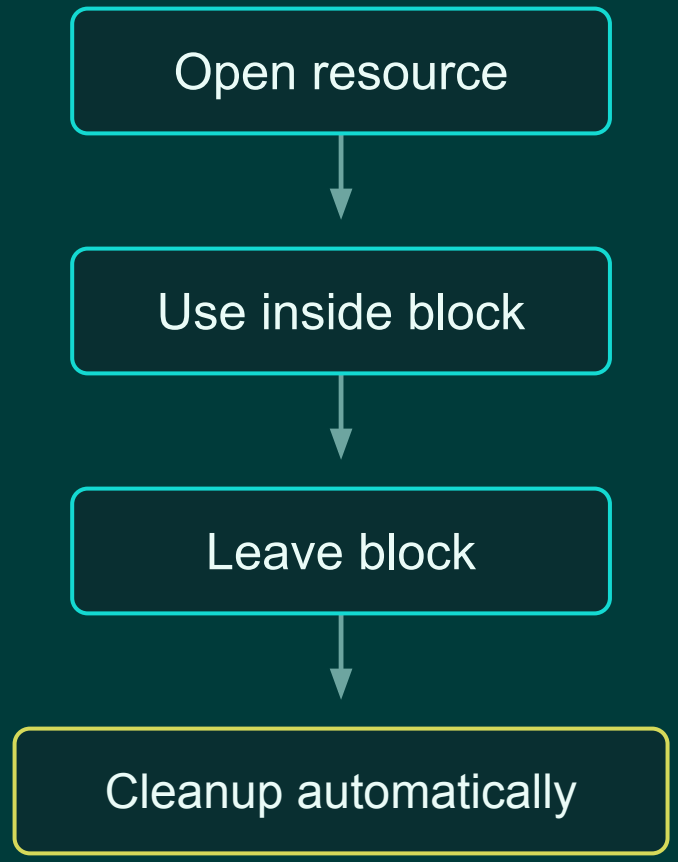
This is why Python has a safer classroom default.

BLOCK 33

The with Pattern

A context manager handles cleanup automatically.

```
with open("data/equipment.txt",
encoding="utf-8") as file:
    contents = file.read()

print(contents)
```



Preferred habit: use with open(...) for file access.

BLOCK 33

Reading the Entire File

Use read() when the whole text is reasonable to hold at once.

```
with open("data/equipment.txt",
encoding="utf-8") as file:
    contents = file.read()

print(contents)
```

contents

```
"EQ-1001
EQ-1002
EQ-1003"
```

read() returns one string containing the file contents.

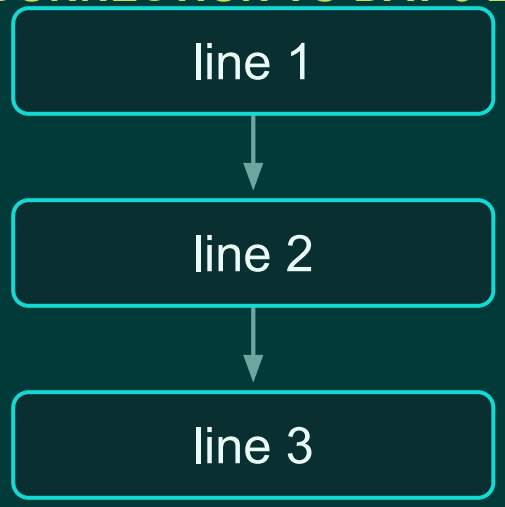
BLOCK 33

Reading Line by Line

A file can be looped over like other iterable values.

```
with open("data/equipment.txt",
encoding="utf-8") as file:
    for line in file:
        print(line)
```

CONNECTION TO DAY 3 LOOPS

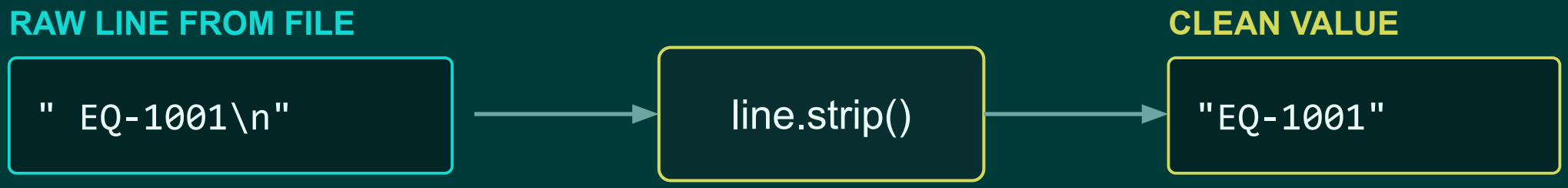


Line-by-line reading is the natural pattern for records stored one per line.

BLOCK 33

Newlines and strip()

Text files include invisible line-ending characters.

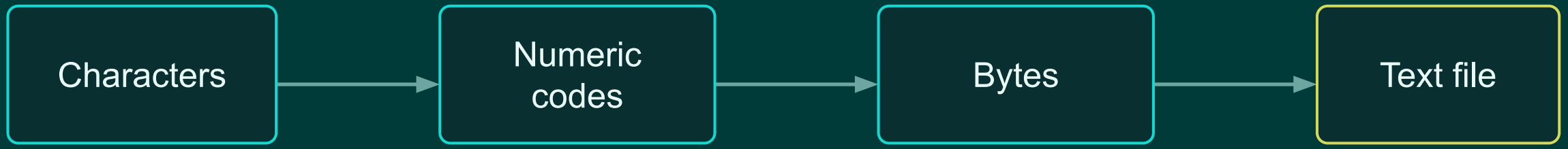



```
with open(file_path, encoding="utf-8") as file:
    for line in file:
        record_id = line.strip()
        print(record_id)
```

BLOCK 33

Text Encoding

A text file stores bytes. Encoding tells Python how to interpret them.



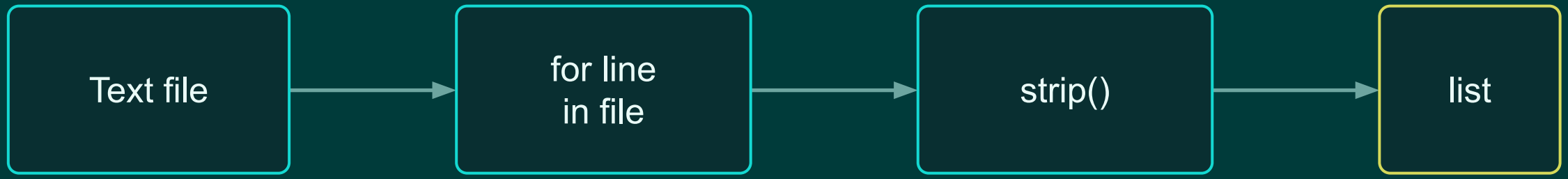

```
with open(file_path, encoding="utf-8") as file:
    contents = file.read()
```

Use UTF-8 explicitly as a safe classroom default.

BLOCK 33

File → Loop → List

The first file-based data pipeline is small but important.




```
record_ids = []

with open(file_path, encoding="utf-8") as file:
    for line in file:
        record_id = line.strip()
        record_ids.append(record_id)
```

BLOCK 33

Common Early Mistakes

Most file bugs are ordinary path, newline, or cleanup problems.

Problem	Symptom	Fix
Wrong CWD	FileNotFoundError	Check relative path
Forgot strip()	Extra blank lines	Use line.strip()
No encoding	Text looks wrong	Use encoding="utf-8"
Manual close	Forgot cleanup	Use with open(...)

Debugging path problems begins with asking: “Where is Python looking?”

BLOCK 33

Guided Exercise Setup

30 minutes: turn a text file into Python data.

STARTING FILE

```
EQ-1001
EQ-1002
EQ-1003
EQ-1004
EQ-1005
```

FILE PATH

```
data/record_ids.txt
```

GOAL

Inspect the path, read the file, loop through lines, and build a list.

Timing: 30 minutes total. Work in small steps and run after each step.

BLOCK 33

Exercise 1: Create and Inspect the Path

Use pathlib to ask questions about the file location.

```
from pathlib import Path

file_path = Path("data/record_ids.txt")

print(file_path.name)
print(file_path.suffix)
print(file_path.parent)
print(file_path.exists())
```

Task

Run the code and explain what each printed value means.

Suggested time: 6 minutes

BLOCK 33

Exercise 2: Read the Entire File

Use with open(...) and read().

```
with open(file_path, encoding="utf-8")
as file:
    contents = file.read()

print(contents)
```

Task

Confirm the file contents match what you expected.

Suggested time: 6 minutes

Question to answer: Is contents one line, many lines, or one string containing newlines?

BLOCK 33

Exercise 3: Read One Line at a Time

The file becomes something you can loop through.

```
with open(file_path, encoding="utf-8") as
file:

    for line in file:
        record_id = line.strip()

        print(record_id)
```

Task

Explain what line.strip() changed.

Suggested time: 8 minutes

BLOCK 33

Exercise 4: Build a List

Convert external text into a Python collection.

```
record_ids = []

with open(file_path, encoding="utf-8") as file:

    for line in file:
        record_id = line.strip()

        record_ids.append(record_id)

print(record_ids)
```

Task

Verify the final value is a list of strings.

Suggested time: 10 minutes

BLOCK 33

Applied Challenge

Clean inconsistent identifiers from a text file.

INPUT FILE

```
 eq-1001
EQ-1002
bad-1003

EQ-1004
 eq-1005
```

RULES

1. Strip whitespace
2. Ignore blank lines
3. Convert to uppercase
4. Accept only EQ- IDs

OUTPUT

valid_ids
invalid_values
processing summary

Timing: 20 minutes. Write small code, run often, inspect values.

BLOCK 33

Challenge Starter Pattern

Do not solve everything at once. Build the pipeline one step at a time.

```
valid_ids = []
invalid_values = []

with open(file_path, encoding="utf-8") as file:
    for line in file:
        value = line.strip().upper()

        if value == "":
            continue

        if value.startswith("EQ-"):
            valid_ids.append(value)
        else:
            invalid_values.append(value)
```

Finish by printing:

```
Valid IDs: 4
Invalid IDs: 1
```

Stretch: print the actual invalid values too.

BLOCK 33

Block 33 Takeaways

Files turn Python programs into tools that work with persistent data.

Path	Identifies a file location
Path.exists()	Checks whether Python can see it
with open(...)	Safely opens and closes files
read()	Returns the whole file as one string
for line in file	Processes the file one line at a time
strip()	Removes newlines and surrounding whitespace

Next: writing files and building persistent output.